ONLINE SHOPPING CART – LOW LEVEL DESIGN

Case study - 2 - Online Shopping Cart

Design the shopping cart module for an ecommerce platform. The system should allow each user to have a shopping cart with possible products / items and quantities. The system should enable adding, removing and updating items (products) and also quantities in the cart at anytime. At any time the system should support view cart checkout workflows for the customer and at anytime before completing the transaction he can go back to the shopping cart. The system should calculate the cart total including offers, discounts and taxes. The system should handle item stock checks and edge cases. At the checkout time before completing order service. The system should connect with payment. Now complete functional, nonfunctional req's, architecture diagram, identify core entities and design low level System prototype.

Functional Requirements

1. Users should be able to add item to cart, remove items from cart, able to update quantity and able to view cart anytime.
2. cart should be able to support multiple items also.
3. The system should check stock availability before checkout
4. The system should be able to calculate the final price if there are any discounts (coupon code).
5. The system should also calculate tax (18%) for every order.

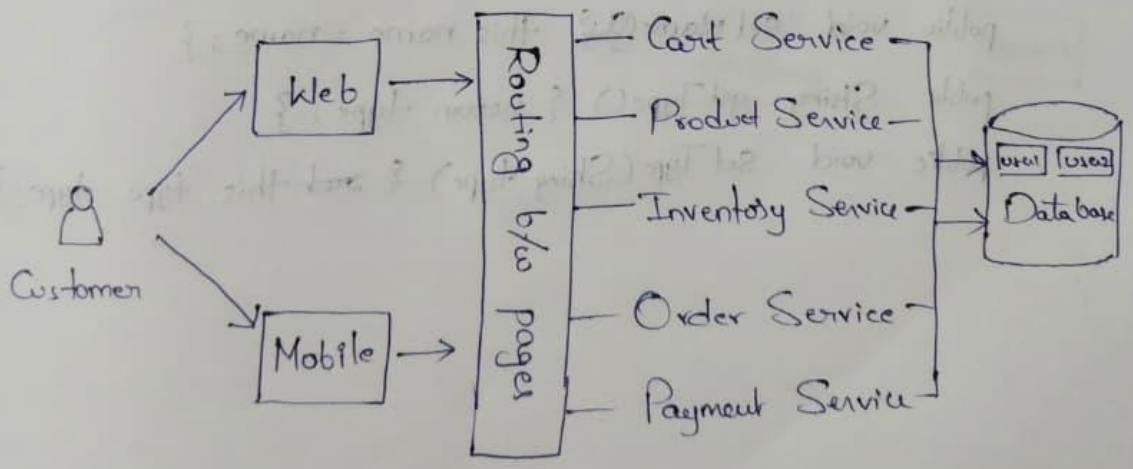
Non Functional Requirements

1. The system will be able to handle millions of carts for millions of customers.
2. Consistency of stock should be maintained because stock must not be overselled.
3. It's better if we maintain multiple payment types.
4. If the payment is failing then there should be option of retrying.

Core Entities

User, Product, Cart, Cart Item, Order, Payment, Inventory.

Architecture



Prototype Code :

Product Class :

```
class Product {
    private int id;
    private String name;
    private String type;
    private double price;
    private int stock;
    public Product(int id, String name, String type, double price, int stock){
        this.id = id;
        this.name = name;
        this.type = type;
        this.price = price;
        this.stock = stock;
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getType() { return type; }
    public void setType(String type) { this.type = type; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
}
```

CartItem Class :

```
class CartItem {
    private Product product;
    private int quantity;
    public CartItem(Product product, int quantity) {
        this.product = product;
        this.quantity = quantity;
    }
    public Product getProduct() { return product; }
    public void setProduct(Product product) { this.product = product; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
    public double getTotalPrice() {
        return product.getPrice() * quantity;
    }
}
```

Cart Class:

```
import java.util.*;

class Cart {
    private int userId;
    private List<CartItem> items;
    public Cart(int userId) {
        this.userId = userId;
        this.items = new ArrayList<>();
    }
    public int getUserId() { return userId; }
    public void setUserId(int userId) { this.userId = userId; }
    public List<CartItem> getItems() { return items; }
    public void setItems(List<CartItem> items) { this.items = items; }
    public void addItem(Product product, int quantity) {
        for (CartItem item : items) {
            if (item.getProduct().getId() == product.getId()) {
                item.setQuantity(item.getQuantity() + quantity);
```

```
            return;
        }
    }
    items.add(new CartItem(product, quantity));
}
public void removeItem(int productId) {
    items.removeIf(item -> item.getProduct().getId() == productId);
}
public void updateQuantity(int productId, int quantity) {
    for (CartItem item : items) {
        if (item.getProduct().getId() == productId) {
            item.setQuantity(quantity);
        }
    }
}
public double calculateTotal() {
    double total = 0;
    for (CartItem item : items) {
        total += item.getTotalPrice();
    }
    return total;
}
}
```

ProductSearch Class :

```
class ProductService {
    private List<Product> products;
    public ProductService(List<Product> products) {
        this.products = products;
    }
    public List<Product> getProducts() {
        return products;
    }
```

```
    public Product[] searchByName(String name) {
        List<Product> result = new ArrayList<>();
        for (Product p : products) {
            if (p.getName().equalsIgnoreCase(name)) {
                result.add(p);
            }
        }
        return result.toArray(new Product[0]);
    }
    public Product[] searchByType(String type) {
        List<Product> result = new ArrayList<>();
        for (Product p : products) {
            if (p.getType().equalsIgnoreCase(type)) {
                result.add(p);
            }
        }
        return result.toArray(new Product[0]);
    }
    public Product[] searchByNameAndType(String name, String type) {
        List<Product> result = new ArrayList<>();
        for (Product p : products) {
            if (p.getName().equalsIgnoreCase(name) &&
                p.getType().equalsIgnoreCase(type)) {
                result.add(p);
            }
        }
        return result.toArray(new Product[0]);
    }
}
```

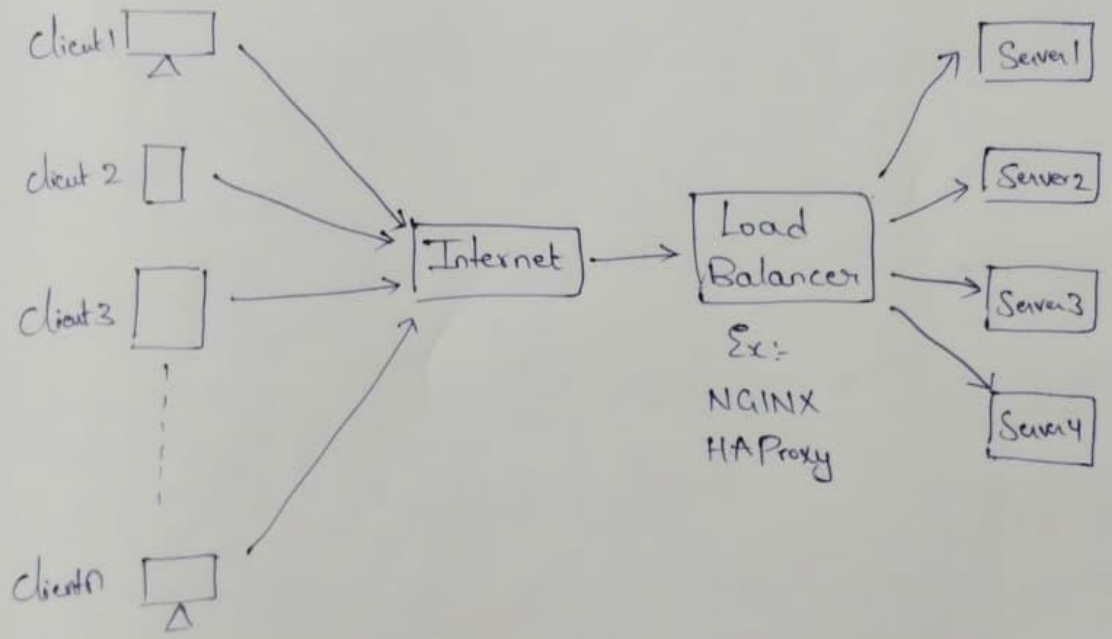
CheckoutService Class :

```
class CheckoutService {
    public boolean validateStock(Cart cart) {
        for (CartItem item : cart.getItems()) {
            if (item.getQuantity() > item.getProduct().getStock()) {
                return false;
            }
        }
        return true;
    }
    public double applyDiscount(double total) {
        if (total > 1000) return total * 0.9;
        return total;
    }
    public double applyTax(double total) {
        return total * 1.18;
    }
    public void checkout(Cart cart) {
        if (!validateStock(cart)) {
            System.out.println("Stock insufficient");
            return;
        }
        double total = cart.calculateTotal();
        total = applyDiscount(total);
        total = applyTax(total);
        Payment payment = new Payment(1, total);
        payment.processPayment();
        System.out.println("Payment Status: " + payment.getStatus());
    }
}
```

Edge Cases

1. If user clicks add to cart many times. It should only update one time.
2. If the quantity of product becomes zero then the system removes that product
3. Product stock = 1 but two users are trying to buy that at same time. (Use Locks)

Load Balancing



Caching :

Storing a copy of data which is accessed frequently in RAM, so that retrieval of data will be faster.

What to Cache in Online Shopping System

1. Storing the cart items in cache will be better.
2. User details like address, personal details.
3. Images of the most popular products should be stored in browser cache.